

Computational Learning Theory

Based on Tom Mitchell's *Machine Learning*, Chapter 7

Contents

1	Introduction to Computational Learning Theory	2
2	Probably Approximately Correct (PAC) Learning	2
2.1	PAC Learning Definition	2
2.2	Probability of Selecting a Bad Hypothesis	2
2.3	Sample Complexity Bound (Finite Hypothesis Space)	3
2.4	Example: Conjunctions of Boolean Literals	3
3	Shattering and the Vapnik-Chervonenkis (VC) Dimension	4
3.1	Shattering a Set of Instances	4
3.2	VC Dimension	4
3.3	Example: VC Dimension of Linear Classifiers	5
4	Sample Complexity using VC Dimension	5
4.1	Interpretation	5
4.2	Example	6
5	The Mistake Bound Model of Learning	6
5.1	Mistake Bound for the Halving Algorithm	6

1 Introduction to Computational Learning Theory

Computational learning theory studies the fundamental capabilities and limitations of algorithms that learn from examples.

It addresses questions such as:

- Under what conditions is successful learning possible or impossible?
- Under what conditions is a learning algorithm guaranteed to learn successfully?
- How many training examples are sufficient to guarantee learning?
- How many mistakes will a learner make before it succeeds?

2 Probably Approximately Correct (PAC) Learning

The PAC learning framework analyzes whether a learner can produce a hypothesis that is approximately correct with high probability.

2.1 PAC Learning Definition

A hypothesis h is considered a good approximation of the target concept if its true error is small.

Two parameters define PAC learning:

- ϵ : accuracy parameter (maximum acceptable error)
- δ : confidence parameter (probability of failure)

A learner PAC-learns a concept if

$$Pr(error_D(h) \leq \epsilon) \geq 1 - \delta$$

This means that with probability at least $1 - \delta$, the learned hypothesis has error at most ϵ .

2.2 Probability of Selecting a Bad Hypothesis

Consider a hypothesis h whose true error exceeds ϵ .

The probability that h correctly classifies one randomly drawn training example is

$$1 - \epsilon$$

The probability that it correctly classifies all m training examples is

$$(1 - \epsilon)^m$$

Using the inequality

$$(1 - \epsilon)^m \leq e^{-\epsilon m}$$

If the hypothesis space contains $|H|$ hypotheses, the probability that at least one bad hypothesis remains consistent with the training data is

$$|H|e^{-\epsilon m}$$

2.3 Sample Complexity Bound (Finite Hypothesis Space)

To ensure PAC learning we require

$$|H|e^{-\epsilon m} \leq \delta$$

Solving for m gives

$$m \geq \frac{1}{\epsilon} \left(\ln |H| + \ln \frac{1}{\delta} \right)$$

This represents the **sample complexity** of learning with a finite hypothesis space.

It specifies the minimum number of training examples required so that, with probability at least $1 - \delta$, the learner returns a hypothesis whose error is at most ϵ .

2.4 Example: Conjunctions of Boolean Literals

Consider learning a concept that classifies emails as **Spam** or **Not Spam**.

Each email is described by three Boolean features:

- x_1 : Contains the word “Free”
- x_2 : Contains the word “Offer”
- x_3 : Contains the word “Win”

The instance space is

$$X = \{0, 1\}^3$$

Each email corresponds to a vector $x = (x_1, x_2, x_3)$.

Target Concept

Assume the true target concept is

$$c(x) = x_1 \wedge x_2$$

An email is spam only if it contains both “Free” and “Offer”.

Hypothesis Space

Suppose the learner uses conjunctions of literals.

For n Boolean attributes

$$|H| = 3^n$$

For $n = 3$

$$|H| = 27$$

PAC Parameters

- $\epsilon = 0.1$
- $\delta = 0.05$

Sample Complexity

$$m \geq \frac{1}{0.1}(\ln 27 + \ln 20)$$

$$\ln 27 \approx 3.296$$

$$\ln 20 \approx 2.996$$

$$m \geq 10(6.292)$$

$$m \approx 63$$

Thus about **63 training examples** are sufficient.

Interpretation

With 63 random training examples:

$$Pr(error \leq 0.1) \geq 0.95$$

So the learner misclassifies at most 10% of emails with 95% confidence.

3 Shattering and the Vapnik-Chervonenkis (VC) Dimension

When the hypothesis space is infinite (e.g., linear classifiers), the previous PAC bound depending on $|H|$ cannot be used.

Instead, learning theory uses the **VC Dimension**.

3.1 Shattering a Set of Instances

A set of instances S is shattered by hypothesis space H if for every possible labeling of S , there exists a hypothesis in H consistent with that labeling.

3.2 VC Dimension

Definition: VC Dimension

The VC dimension of hypothesis space H , denoted $VC(H)$, is the size of the largest set of instances that can be shattered by H .

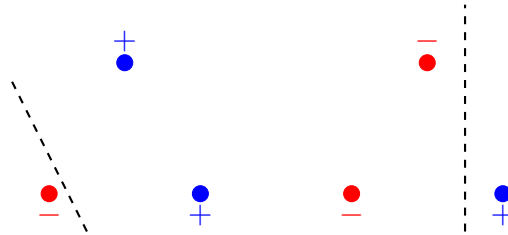
3.3 Example: VC Dimension of Linear Classifiers

A linear classifier in two dimensions (a straight line) can shatter any set of 3 points but cannot shatter 4 points (because of the XOR configuration).

Thus

$$VC(H) = 3$$

Shattering 3 Points with a Linear Classifier



4 Sample Complexity using VC Dimension

When H is infinite, the number of examples required depends on $VC(H)$ instead of $|H|$.

Let $VS_{H,D}$ denote the **version space** (all hypotheses consistent with training data).

We ask:

How many random training examples are needed so that every hypothesis remaining in the version space has error at most ϵ with probability at least $(1 - \delta)$?

According to VC theory (Mitchell, Chapter 7):

$$m \geq \frac{1}{\epsilon} \left(4 \log_2 \left(\frac{2}{\delta} \right) + 8VC(H) \log_2 \left(\frac{13}{\epsilon} \right) \right)$$

where

- ϵ : maximum allowable generalization error
- δ : probability that the guarantee fails
- $VC(H)$: VC dimension

This guarantees that with probability at least $1 - \delta$, every hypothesis in the version space has error at most ϵ .

4.1 Interpretation

$$m \propto \frac{VC(H)}{\epsilon}$$

Thus more expressive models require more training data.

4.2 Example

For linear classifiers in 2D

- $VC(H) = 3$
- $\epsilon = 0.05$
- $\delta = 0.05$

Using the VC bound we can estimate the required number of training examples.

Key Insight

The number of required training examples grows approximately linearly with the VC dimension.

5 The Mistake Bound Model of Learning

Instead of asking how many examples are needed to achieve a certain error rate, the Mistake Bound model asks:

How many mistakes will the algorithm make before learning the concept perfectly?

5.1 Mistake Bound for the Halving Algorithm

The Halving algorithm maintains the version space VS .

For each new instance it predicts using the majority vote of hypotheses in VS .

If a mistake occurs, at least half of the hypotheses are eliminated.

Thus the number of mistakes satisfies

$$M_{Halving} \leq \log_2(|H|)$$

